

RADlab Advanced Instrument Brief

For students prototyping a custom questionnaire with an AI assistant.

How to use this document (read this part yourself)

Some instruments — demographics surveys, censuses, anything with dropdowns, free text, multi-select, or "if yes, then ask..." branching — can't be expressed in the platform's standard questionnaire JSON. They have to be coded into the platform by the lab. Your job is **not** to do that integration. Your job is to produce a **prototype package** that makes the lab's integration work fast and unambiguous.

1. Attach this entire document to a conversation with an AI assistant (e.g. Claude).
2. Also give it your source material: the questionnaire text, the paper it comes from, or your own draft of the questions.
3. Ask it to produce the two deliverables described below, following every convention in this brief.
4. Review the mockup yourself — open the HTML file in a browser, click through every question, and check the wording against your source. You are the content expert; the AI will follow this brief's *style*, but only you can verify the *substance*.
5. Hand both files to the lab.

Everything below this line is written for the AI assistant. You should read it too — it explains what you'll get back.

Instructions for the AI assistant

You are helping a student prototype a custom ("advanced") instrument for **RADlab's Come, See platform** — a React web platform that delivers psychology studies (games, questionnaires, forms) to research participants. The student will hand your output to the lab, which will port it to a production React component. Your output is a *prototype and specification*, not production code.

Produce exactly two deliverables

Deliverable A — a content specification (markdown). The complete, unambiguous definition of the instrument: every question, every option, all branching logic, and the shape of the stored response. Format defined in §3.

Deliverable B — a self-contained HTML mockup (one `.html` file). A clickable, working prototype of the full instrument that looks and behaves like the platform's real instruments. Requirements in §4–§6.

Do **not** produce: a React app or component, anything with a build step or external JS dependencies, database code, Supabase calls, or SQL. The lab writes all of that from your spec.

§1. What an advanced instrument is on this platform

The platform has two questionnaire tiers:

- **Standard (JSON)** — Likert-scale and checklist instruments defined as pure data and played by a shared renderer. If the instrument is *only* rating scales with numeric anchors, stop and tell the student it belongs in the standard tier instead — there is a separate conversion workflow for that, and it is much less work for everyone.
- **Advanced (coded)** — everything else: mixed input types (free text, number entry, radio groups, multi-select checkboxes, ladders/sliders), conditional follow-ups, inline definitions, "please specify" boxes. Each one is a bespoke React component the lab maintains.

An advanced instrument renders as **one scrollable page of numbered sections** (not one question per screen), with a single submit button at the bottom. The platform's reference implementations are its Standard Demographics form and the U of T Student Equity Census; the conventions below are extracted directly from those.

§2. Ethics and wording conventions (non-negotiable)

These come from the lab's research-ethics practice. Apply them even if the student's source material doesn't:

1. **Every sensitive question offers "Prefer not to answer"**. Demographic identity questions (gender, sexuality, disability, race/ethnicity, religion, income, etc.) must include it as an option. Flag any question in the source that lacks it.
2. **"Required" means answered, not disclosed**. All questions are required, but "Prefer not to answer" always satisfies the requirement. The submit button stays disabled until every question has *some* answer. Below a disabled submit button, show the hint:

"All questions are required — choose "Prefer not to answer" on any you wish to skip."

3. **In a multi-select, "Prefer not to answer" is exclusive**. Selecting it clears every other selection in that question; selecting anything else clears it.
4. **Open with a confidentiality preamble**. One short paragraph under the title stating what the questions are for and that responses are confidential. Example: *"These questions help us understand our participants. All responses are confidential."*
5. **Gate sensitive follow-ups behind a yes/no question**. Don't show a disability-type checklist to everyone; ask "Do you identify as a person with a disability?" first and reveal the checklist only on "yes". Same pattern for Indigenous identity, diagnosis lists, etc.
6. **Define terms inline, not in a glossary**. If an option or question uses a term participants may not know (e.g. "Two-Spirit", "racialized"), put a short definition in smaller muted text directly under the option label or question, not on a separate page.
7. **Optional questions are labelled optional**. If a question is genuinely optional (e.g. a free-text feedback box), mark its heading with "(optional)" in muted text and exclude it from the submit gate.
8. Preserve the source instrument's wording faithfully. If wording seems outdated or problematic, keep it, complete the prototype, and add a note in the spec's open-questions section — the decision to alter a published instrument belongs to the lab, not the prototype.

§3. Deliverable A — content specification format

A markdown document with these sections, in order:

3.1 Header block

- # <Instrument name> — content specification
- Proposed key: <snake_case identifier, e.g. `sleep\_history`>
- Source: <citation, URL, or "author-drafted">
- Prepared by: <student name> with AI assistance, <date>
- Status: prototype for lab review — not integrated

3.2 Question table

One numbered row per question. For each:

#	Question text (verbatim)	Input type	Options (value → label)	Shown when	Required
---	-----------------------------	------------	----------------------------	------------	----------

- **Input type** is one of: number entry, free text (single line), free text (multi-line), single select — button row (≤5 short options), single select — radio column (longer lists), multi-select checkboxes, multi-select with hierarchical sub-options, ladder/ discrete scale. If the instrument truly needs something outside this set, describe it precisely and flag it in open questions.
- **Options** listed as snake_case_value → "Participant-facing label". Mark exclusive options (exclusive), options that reveal a text box (specify), and options with inline definitions (def: ...).
- **Shown when** is always or a condition on a previous answer, e.g. Q3 = yes.

3.3 Response payload

The exact JSON object one completed submission produces. Rules in §6; show a filled example with realistic values, not a schema.

3.4 Open questions for the lab

Anything unresolved: wording concerns, options you weren't sure about, branching ambiguity in the source, scoring rules if any. An empty section is fine; a hidden assumption is not.

§4. Deliverable B — mockup ground rules

- **One .html file, fully self-contained.** Inline <style> and vanilla inline <script> only. No CDN scripts, no frameworks, no build step. A Google Fonts <link> for the two brand fonts is allowed *with* system fallbacks, since some preview environments block external requests (the font stacks in §5 degrade gracefully).
- **Fully interactive.** Every gate reveals/hides its follow-up, exclusive options clear their siblings, specify boxes appear on selection, and the submit button enables only when the §2 requirement gate passes.
- **Submit proves the payload.** Clicking the enabled submit button must not go anywhere; instead show a "Preview complete — nothing was saved" panel that displays the §3.3 response JSON built from the actual current selections. This is how the student and lab verify the data shape matches the spec.

- **Banner at the top of the page:** a small muted strip reading

PROTOTYPE – <instrument name> – not connected to the platform so the file can never be mistaken for a live instrument.

- Mobile-friendly: single column, max-width: 640px, readable at 375px wide.

§5. Design system

The mockup must look native to the platform. Reproduce these tokens and patterns exactly.

5.1 Tokens

Define these CSS custom properties on `:root` and use them throughout (they mirror the platform's real values):

```
:root {
  --bg:      #FCF0F5;          /* page background – warm pink-tinted off-white */
  --pk:      #f068a4;          /* brand pink – selected states, submit button */
  --pkb:     rgba(240, 104, 164, 0.18); /* pink wash – selected option background */
  --tx:      #1c1c1e;          /* primary text */
  --tx2:     #6b6c70;          /* secondary text – preambles, definitions */
  --tx3:     #abadb0;          /* muted text – hints, "(optional)" */
  --bd:      rgba(180, 100, 140, 0.13); /* borders – hairline pink-grey */
  --err-tx:  #A32D2D;          /* error text */
}
```

Inputs and buttons sit on `#fff`; the page itself is `--bg`.

5.2 Typography

```
/* headings – instrument title and numbered section headings */
font-family: "DM Serif Display", Georgia, serif; /* weight 400, never bold */
/* everything else */
font-family: "DM Sans", system-ui, sans-serif;
```

Optional (degrades gracefully if blocked):

```
<link href="https://fonts.googleapis.com/css2?family=DM+Serif+Display&family=DM+Sans:wght@400;600&display=swap" rel="stylesheet">
```

Sizes: page title 32px; section heading 21px; question label 15px, weight 600; option label 14px; definition/preamble text 12.5–14px in `--tx2/--tx3`; line-height ~1.5–1.6 on all prose.

5.3 Page layout

- Column: max-width: 640px; margin: 0 auto; padding: 40px 24px 80px;
- Vertical rhythm: sections in a flex column with gap: 44px; inside a section, gap: 12px.
- Section headings numbered: 1. Gender Identity, 2. Sleep History, ... Optional sections get

`<span> "(optional)"` in 13px `--tx3` DM Sans.

- Title, then preamble paragraph (14px, `--tx2`), then the sections, then error line (if any), then submit button, then the requirement hint under it (13px, `--tx3`).

5.4 Component patterns

Single select, few short options — button row. Horizontal wrap of pill buttons:

```
.opt-btn      { padding: 10px 20px; border-radius: 10px; border: 1px solid var(--bd);
                background: #fff; font-size: 14px; color: var(--tx2); cursor: pointer;
                transition: all 0.12s; }
.opt-btn.sel  { background: var(--pkb); color: var(--pk); border-color: var(--pk);
                font-weight: 600; }
```

Single select, long list — radio column. Native `<input type="radio">` with `accent-color: var(--pk)`, one per row; rows are `display:flex; gap:10px; padding:7px 10px; border-radius:8px;` with the label at 14px `--tx`.

Multi-select — checkbox column. Same row styling with `<input type="checkbox">`, `accent-color: var(--pk)`. Inline definitions render as a block under the label: 12.5px, `--tx3`, margin-top 2px.

Hierarchical multi-select. Checking a parent reveals an indented child column: `margin-left: 28px; padding-left: 12px; border-left: 2px solid var(--bd);`. Unchecking the parent removes its children from the selection.

"Please specify" box. Appears only while its option is selected; a text input indented under the option: `max-width: 420px; margin: 4px 0 6px 30px; padding: 8px 12px; border: 1px solid var(--bd); border-radius: 8px; background: #fff; placeholder "Please specify..."`.

Definition block (for a definition attached to a whole question rather than one option): 13px `--tx2` text in a panel — `background: var(--bg); border: 1px solid var(--bd); border-radius: 10px; padding: 12px 14px;`.

Free text. Single line: styled like the specify box at full column width. Multi-line: `<textarea rows="4">` with `border-radius: 10px; resize: vertical;`.

Number entry. Same input styling, `type="number"`, sensible `min/max`, short placeholder like "e.g. 24".

Submit button.

```
.submit { background: var(--pk); color: #fff; border: none; border-radius: 10px;
          padding: 13px 32px; font-size: 16px; font-weight: 600; }
.submit:disabled { opacity: 0.4; cursor: default; }
```

Label: Continue → (the platform reserves "Submit" wording for admin tools).

Error line. 14px `--err-tx`, e.g. "Could not save — please try again." (in the mockup this never actually triggers; include the style anyway so the lab sees intended placement, directly above the submit button).

§6. Data conventions

The response payload (spec §3.3, and what the mockup's preview panel prints) follows the platform's storage pattern — one self-describing JSON object per completed submission:

- **Keys and option values are** `snake_case`, `prefer_not_to_answer`, not "Prefer not to answer"; the participant-facing label is presentation only.
- **Single select** → a string value, or `null` if the instrument allows skipping (it normally doesn't — see §2.2).

- **Multi-select** → an array of value strings. "Prefer not to answer" appears as `["prefer_not_to_answer"]` alone, never alongside others.
- **Hierarchical multi-select** → parent values plus `"parent:child"` compound keys in one flat array, e.g. `["asian", "asian:east_asian"]`.
- **"Please specify" text** → a sibling key named `<field>_other`, which is the entered string when the specify option is selected and `null` otherwise. Never fold free text into the option array.
- **Gated follow-ups** → when the gate answer hides the follow-up, the follow-up's key is still present: `[]` for multi-selects, `null` for scalars. Every key appears in every payload.
- **Optional free text** → trimmed string, or `null` when empty.
- **Numbers as numbers**, not strings (`"age": 24`).

Example fragment:

```
{
  "disability": "yes",
  "disability_types": ["adhd", "not_listed"],
  "disability_types_other": "dysgraphia",
  "religion": ["prefer_not_to_answer"],
  "religion_other": null,
  "feedback": null
}
```

§7. Hand-back checklist (for the student, before sending to the lab)

- Opened the HTML file in a browser; every question renders and every gate/specify/exclusive behavior works.
- Clicked through a full run and compared the preview panel's JSON to the spec's §3.3 payload — keys, casing, and shapes match.
- Checked every question's wording against the source instrument, word for word.
- Every sensitive question has "Prefer not to answer", and it is exclusive in multi-selects.
- The spec's open-questions section says everything you're unsure about (or is genuinely empty).
- Sending **both** files: the spec (`.md`) and the mockup (`.html`).

§8. What happens after hand-off (context, no action needed)

The lab ports the prototype to a React component in the platform codebase, registers it in the advanced-instruments registry, creates a database table with row-level security, and wires it into the study session builder. Your spec's question table and payload example are what make that port mechanical instead of interpretive — which is why §3 and §6 matter as much as the mockup looking right.